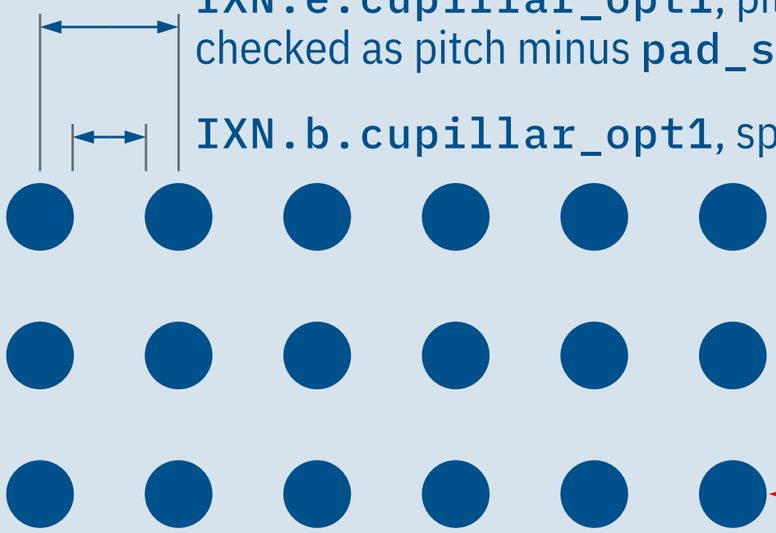
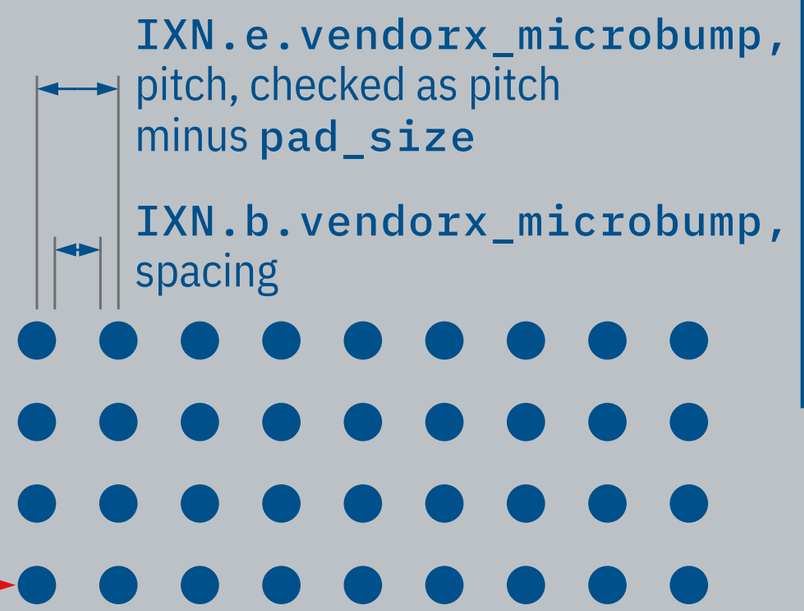
## `ixn_region` = `interconnect_region` if an adapter narrowed it, else `chiplet_attachment_input`

**U1** boundary,
from the manifest `polygon_dbu`

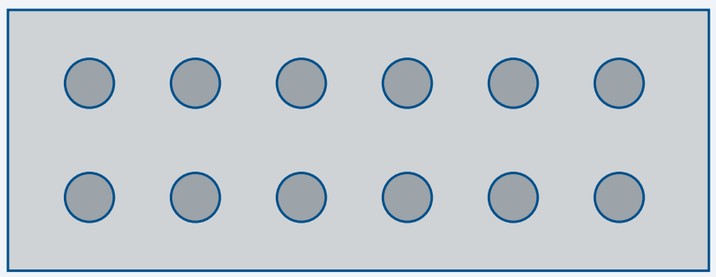



`ixn_method_regions['cupillar_opt1']`
`= ixn_region AND U1 boundary`

**U2** boundary,
from the manifest `polygon_dbu`




`ixn_method_regions[`
`'vendorx_microbump']`
`= ixn_region AND U2 boundary`



`ixn_region` minus `ixn_claimed`, reported as `IXN.b.unclaimed` and `IXN.e.unclaimed`; each method's pads against them: `IXN.x.<method>.unclaimed`

checked with the adapter's assembly-global numbers, and only if an interconnect adapter was supplied; without an interconnect adapter, the deck reports each one as `IXN.unassociated`: not checked, not a spacing violation

`IXN.x.cupillar_opt1.vendorx_microbump`, evaluated at max of the two `IXN_spacing` values

without this the gap between dies of different methods is unchecked; each method region contains only its own pads

**LEGEND**

- method region, `cupillar_opt1`
- method region, `vendorx_microbump`
- die boundary polygon from the boundary manifest
- unclaimed remainder, covered by no method region
- `ixn_region` outline
- cross-method spacing check

every spacing evaluation subtracts a 10 nm circle tolerance, because a 256-vertex polygon snapped to a 1 nm grid measures a few nanometres short of a true circle